

Fundamentos: tipos, operadores y casting

No.	Respuesta	Explicación:
1	d. Ofrecer herramientas necesarias para compilar, depurar y ejecutar aplicaciones Java.	JDK (Java Development Kit) es el kit que usamos para desarrollar en Java y trae herramientas como para compilar el código, para depurarlo y para empaquetarlo.
2	e. jar	La herramienta jar sirve específicamente para crear archivos .jar, que nos permiten empaquetar clases y recursos de una aplicación.
3	d. jconsole	jconsole se utiliza para conectarse a una JVM y observar información como memoria, hilos y clases.
4	d. boolean j = (1 < 5);	En Java un boolean solamente puede tener true o false. Como 1 < 5 es una comparación y su resultado es true. En Java no podemos usar 0 o 1 como boolean, y tampoco null porque boolean es un primitivo.
5	d. boolean c = (7 >= 7); e. boolean e = (10 > 5 && 2 < 1); f. boolean a = (3 < 6);	7 >= 7 devuelve true, 10 > 5 && 2 < 1 devuelve false y 3 < 6 devuelve true; las tres son expresiones booleanas. En cambio, 4 - 14 produce un int, "true" es un String y 15 = 2 no es una comparación válida.
6	e.double	Cuando hacemos una operación entre un double y un short, Java convierte el short a double para poder hacer la operación.
7	b.1 2	Primero a++ aumenta a de 0 a 1. Después println(a++) imprime primero el valor actual, que es 1, y después aumenta a a 2. Finalmente se imprime 2.
8	c.22	b empieza en 4 y b-- lo deja en 3. Después --b primero disminuye b a 2 y luego imprime ese valor. El siguiente println(b) vuelve a imprimir 2, por eso sale 22
9	c.true	Primero se evalúa a && b porque && tiene mayor prioridad que . Entonces queda (true && false) true, que es false true, y finalmente obtenemos true.
10	b.false	Primero se calcula x && y, que es true && false, dando false. Después !x convierte true en false, así que tenemos false false, cuyo resultado es false
11	b.Result: 35 Result: 8	En la primera línea, como ya empezamos con un String, los + siguientes funcionan como concatenación, por eso queda 35. En la segunda línea los paréntesis hacen que primero se calcule 3 + 5, dando 8, y después se concatena con "Result:".
12	c.Result: 235 Result: 215	La concatenación se hace de izquierda a derecha. Por eso en la primera línea se obtiene 235. En la segunda, 3 * 5 tiene prioridad y se calcula primero, dando 15; después se concatena, formando 215.

13	13	El operador entre enteros hace un OR bit a bit. 9 en binario es 1001 y 4 es 0100; al hacer OR obtenemos 1101, que en decimal es 13.
14	a.boolean flag = Boolean.FALSE; b.boolean flag = true; d.boolean flag = false;	true y false son valores booleanos válidos. Boolean.FALSE también funciona porque Java puede convertir automáticamente el objeto Boolean a un boolean mediante unboxing. 0 es un int y "false" es un String, por lo que no sirven directamente.
15	a.&& b. d. e.==	&& y son operadores lógicos, también puede trabajar con booleanos y == sirve para comparar dos valores booleanos. El operador + no está definido para valores booleanos.
16	b.Line 16. d.Line 18	int a, b, c; y int d, e, f; son declaraciones normales de varias variables del mismo tipo. Int con mayúscula no existe como tipo primitivo, y tampoco podemos volver a escribir int después de una coma en la misma declaración.
17	e. No puedes asignar un byte a un carácter.	Un byte y un char tienen características diferentes: byte es con signo y char no. Por eso Java no permite convertir automáticamente una variable byte normal a char; necesitaríamos un cast como (char) b.
18	b.three	Para que Java reconozca el método main del programa necesitamos public static void main(String[] args). El método que imprime three tiene exactamente esa estructura; los otros métodos tienen parámetros o nombres diferentes y no son el main.
19	c.class Fred1 { public static void main(String[] args) { System.out.println(args); } } d.class Fred1 { public static void main(String[] args) { System.out.println(args[1]); } }	Al ejecutar el programa tenemos dos argumentos: hello está en args[0] y walls en args[1]. La opción c imprime el arreglo completo y la d imprime args[1], que sería walls. La opción a intenta acceder a args[2], que no existe, y la b recibe un String en lugar de un arreglo.
20	d.long	l es long, así que cuando hacemos l * 10, el resultado es long. Después s / (l * 10) también produce un long. Por eso necesitamos como mínimo long; float y double también podrían recibirlo, pero son tipos más grandes.
21	d.long	Se repite la pregunta de arriba.
22	b.float	La operación produce un long, y Java permite convertir automáticamente un long a float o double. Como la pregunta pide específicamente un tipo de punto flotante y float es más pequeño/restringido que double, la respuesta es float.

23	c.public static void main(String... String) {} d.public static void main(String[] args) {}	El main debe ser public static void y recibir String[] o String....
24	b.The java -jar command ignores the CLASSPATH environment variable and command-line classpath settings. d.The java -jar command requires the JAR file to have a META-INF/MANIFEST.MF file specifying the main class. e.The java -jar command can only be used to execute JAR files that are executable JARs, i.e., those that contain the META-INF/MANIFEST.MF file with the Main-Class attribute.	Cuando usamos java -jar, Java busca dentro del JAR el archivo META-INF/MANIFEST.MF y necesita que este indique cuál es la clase principal mediante Main-Class. Además, el classpath indicado mediante -cp o -classpath y la variable CLASSPATH no se utilizan de la misma forma en este modo, porque el JAR define su propio classpath